INSTITUTO UNIVERSITÁRIO DE LISBOA

Mestrado em Engenharia Informática

**Segurança em Redes e Sistemas de Informação**

# Análise de Vulnerabilidades Web e Mitigação com *WAF ModSecurity*


**Autores:**

Daniel Nunes, Nº 123547

Guilherme Alves, Nº 122661

João Oliveira, Nº 142584


**Docente:** Prof. Daniel Caçador

**Data:** 17 de outubro de 2026

## Resumo

Este documento descreve o impacto de vulnerabilidades Web críticas, como *SQL Injection* (SQLi) e *Cross-Site Scripting* (XSS) em aplicações Web. Também analisa a eficácia da mitigação utilizando o *Web Application Firewall* (WAF) *ModSecurity* com o OWASP *Core Rule Set* (CRS). Na fase teórica deste projeto, é feito o estudo detalhado sobre o OWASP Top 10, compara as principais soluções WAF disponíveis no mercado com as abordagens encontradas no mundo científico, e define um modelo matemático para avaliar o risco, precisão de detaçã e latência de processamento.



**Palavras-chave:** Segurança Web, OWASP Top 10, WAF, *ModSecurity*, *SQL Injection*, XSS.

# Índice

**Resumo** **1**

**Glossário** **3**

**1 Introdução** **4**

**2 Estado da Arte** **5**

2.1 Paradigma da Segurança na Camada Aplicacional . . . . . 5

2.2 Mecânica das Vulnerabilidades Críticas (OWASP) . . . . . 5

2.3 Modelos de Deteção e Mecanismos de Inspeção em WAFs . . . . . 6

2.4 Arquitetura do *ModSecurity* v3 e OWASP *Core Rule Set* . . . . . 7

**3 Comparação de Soluções WAF e Trabalhos Relacionados** **8**

3.1 Comparação de Soluções WAF . . . . . 8

3.2 Trabalhos Relacionados na Literatura . . . . . 9

**4 Formulação Teórica e Métricas Matemáticas** **10**

4.1 Métricas de Deteção . . . . . 10

4.2 Métricas de Desempenho e Latência . . . . . 10

**5 Metodologia e Repositório Online** **11**

5.1 Ambiente de Prova de Conceito . . . . . 11

5.2 Repositório de Documentação (GitHub) . . . . . 11

**6 Conclusões e Trabalho Futuro** **12**

## Glossário

**CRS** *Core Rule Set* (conjunto de regras da OWASP para WAFs).

**DVWA** *Damn Vulnerable Web Application.*

**FN** Falso Negativo (*False Negative*).

**FP** Falso Positivo (*False Positive*).

**OWASP** *Open Web Application Security Project.*

**SQLi** *Structured Query Language Injection.*

**TN** Verdadeiro Negativo (*True Negative*).

**TP** Verdadeiro Positivo (*True Positive*).

**WAF** *Web Application Firewall.*

**XSS** *Cross-Site Scripting.*

# 1 Introdução

O crescimento constante de aplicações Web deixas as infraestruturas expostas a ameaças cada vez mais avançadas. Falhas de segurança na camada de aplicação são frequentemente exploradas porque estão diretamente acessíveis à rede pública e porque o desenvolvimento seguro é complexo.

Este projeto pretende analisar o mecanismo de proteção oferecido por uma *Web Application Firewall* (WAF), com foco no *ModSecurity* v3, utilizando o OWASP *Core Rule Set* (CRS) integrado no servidor web Nginx.

A estrutura deste documento tem a Secção 2 com o estado da arte sobre vulnerabilidades críticas e o ecossistema de WAFs. A Secção 3 apresenta uma análise comparativa de soluções existentes e de trabalhos relacionados. A Secção 4 define matematicamente as métricas de validação e risco. A Secção 5 descreve a arquitetura proposta e inclui o repositório do projeto. A Secção 6 traz as conclusões iniciais e o trabalho futuro.

# 2 Estado da Arte

## 2.1 Paradigma da Segurança na Camada Aplicacional

A consolidação de arquiteturas orientadas a serviços e aplicações Web fez com que o protocolo HTTP/HTTPS se tornasse o principal canal de comunicação entre sistemas empresariais. No entanto, as soluções tradicionais de segurança demonstram ser ineficientes diante de ataques que envolvam diretamente a camada de aplicação.

Isto acontece porque quer o tráfego, quer as tentativas de invasão passam pelas mesmas portas padrão (80 e 443), e não podem ser diferenciados apenas pelo endereço ou porta. Por isso, é essencial escolher mecanismos que analisem profundamente o conteúdo dos pedidos e respostas.

Os WAFs têm um papel fundamental na estratégia de defesa em profundidade. Estes não substituem as boas práticas de desenvolvimento seguro de software, atuam como uma camada de proteção adicional, ajudando a mitigar as ameaças de forma imediata enquanto as vulnerabilidades das aplicações em produção ainda não foram corrigidas.

## 2.2 Mecânica das Vulnerabilidades Críticas (OWASP)

O acordo OWASP documenta e atualiza frequentemente os riscos de maior incidência que afetam aplicações Web. São abordadas duas das principais categorias do OWASP Top 10, explorando o seu funcionamento e os desafios de mitigação:

- ***SQL Injection* (SQLi):** Acontece quando entradas erradas feitas pelo utilizador são concatenadas de forma direta em instruções SQL sem a devida validação de tipos. Esta falha viola a fronteira entre o plano de controlo e dos dados da base de dados. Estas explorações podem ser:

    1. *In-Band:* Quando o atacante utiliza o operador `UNION` para juntar o resultado das suas consultas maliciosas à resposta original da aplicação.
    2. *Inferencial (Blind):* Quando a aplicação oculta mensagens de erro e resultados, forçando o atacante a presumir a informação através de respostas booleanas (*Boolean Blind*) ou medição do tempo de latência forçado na base de dados (*Time-Based Blind*).
    3. *Out-of-Band:* Quando a extração de dados é redirecionada através de outros canais e protocolos, designadamente pedidos DNS ou HTTP externos desencadeados pelo servidor de base de dados.

- **Cross-Site Scripting (XSS):** Caracteriza-se pela falha na higienização e codificação de caracteres especiais nas respostas devolvidas ao cliente. Pode ser divido em:

    1. *Non-Persistent:* O *script* malicioso é integrado no URL ou pedido HTTP e reflete diretamente pelo servidor Web para o utilizador
    2. *Persistent:* O conteúdo malicioso é salvo permanentemente no servidor alvo e é entregue a todos os utilizadores que carreguem a página afetada.
    3. *DOM-based:* A vulnerabilidade existe inteiramente no lado do cliente ao invés de estar no código dentro do servidor. O *browser* processa dados não confiáveis do DOM de maneira insegura.

## 2.3 Modelos de Deteção e Mecanismos de Inspeção em WAFs

Os motores de inspeção de WAF assentam tipicamente em dois paradigmas fundamentais de segurança:

- ***Whitelisting* (Modelo Positivo):** Estabelece uma estratégia de segurança baseada no principio do menor privilégio. Ao invés de estar continuamente a pesquisar por assinaturas maliciosas, define o que um tráfego legítimo é e deixa-o passar.
- ***Blacklisting* (Modelo Negativo):** Baseia-se no princípo de bloquear o que é explicitamente conhecido como malicioso. Atua como um filtro que analisa o tráfego em busca de ameaças já conhecidas e permite a passagem de tudo o resto por padrão.

Para mitigar a fuga de atacantes, os motores de WAF modernos aplicam sequências de transformação e normalização de dados antes de submeter os conteúdos à avaliação de regras.

## 2.4 Arquitetura do *ModSecurity* v3 e OWASP *Core Rule Set*

O *ModSecurity* consolidou-se como o motor de WAF em código aberto de maior relevância. A versão 3 foi reformulada sob uma arquitetura modular em C++, desacoplando o núcleo de processamento do servidor aplicacional, permitindo a integração fluida com o servidor Nginx.

O ciclo de vida de cada transação HTTP decompõe-se em cinco fases sequenciais:

1. **Fase 1 (*Request Headers*):** Análise inicial dos cabeçalhos do pedido antes do processamento do corpo.
2. **Fase 2 (*Request Body*):** Validação do corpo da mensagem HTTP. É bastante crítica pois são avaliados os parâmetros submetidos por formulários e *payloads*.
3. **Fase 3 (*Response Headers*):** Avaliação dos cabeçalhos devolvidos pelo servidor de aplicação.
4. **Fase 4 (*Response Body*):** Inspeção do conteúdo da resposta, prevenindo a transferência de dados sensíveis ou mensagens de erro interno.
5. **Fase 5 (*Logging*):** Registo estruturado de auditoria e telemetria no ficheiro de eventos para análise.

Em conjunto com o OWASP *Core Rule Set* (CRS), o ModSecurity implementa preferencialmente o modo de pontuação de anomalias. Neste modelo, uma regra ativada não provoca o encerramento imediato da ligação, em alternativa, atribui um valor incremental de risco consoante a severidade da ocorrência. O bloqueio efetivo só ocorre no termo da Fase 2 ou Fase 4 caso o somatório das pontuações acumuladas ultrapasse o patamar de bloqueio pré-configurado, reduzindo a taxa de falsos positivos face a métodos estáticos de correspondência imediata.

# 3 Comparação de Soluções WAF e Trabalhos Relacionados

## 3.1 Comparação de Soluções WAF

A Tabela 1 mostra as principais características de diferentes abordagens de WAF existentes.

Tabela 1: Comparação entre diferentes soluções de WAF.

| **Solução** | **Tipo de Licença** | **Modelo de Deteção** | **Vantagens / Limitações** |
|---|---|---|---|
| **ModSecurity + CRS** | Open-Source (Apache 2.0) | Assinaturas e *Anomaly Scoring* | Alta flexibilidade e personalização de regras, requer calibração contra falsos positivos. |
| **AWS WAF** | Proprietário (Cloud) | Regras geridas e regras customizadas | Totalmente integrado na AWS e escalável, custos associados por regras e tráfego. |
| **Cloudflare WAF** | Proprietário (SaaS) | Inteligência de ameaças global + ML | Baixa manutenção e proteção DDoS incluída, menor visibilidade dos logs internos detalhados. |
| **NAXSI** | Open-Source (GPLv3) | Modelo positivo / Heurística | Extremamente leve para Nginx, requer processo complexo de aprendizagem inicial (*whitelisting*). |

## 3.2 Trabalhos Relacionados na Literatura

A eficácia dos WAFs na mitigação de ataques à camada de aplicação tem sido amplamente investigada pela comunidade científica, com foco nos limites dos motores baseados em assinaturas e na sobrecarga de processamento associada.

Estudos de referência na avaliação de eficácia defensiva demonstram que, embora o *ModSecurity* com o OWASP *Core Rule Set* (CRS) proporcione uma barreira robusta contra ataques padrão, a dependência de regras estáticas torna o sistema vulnerável a técnicas avançadas de *fuzzing* de *payloads*. Investigações recentes evidenciam que pequenas alterações sintáticas em comandos de *SQL Injection* e *Cross-Site Scripting* conseguem, em determinados cenários, contornar as expressões regulares do CRS sem perder a sua capacidade intrusiva na base de dados, provocando taxas indesejadas de falsos negativos.

Diversos autores analisaram o compromisso entre segurança e usabilidade operacional. O aumento do nível de paranoia nas regras do OWASP CRS eleva consideravelmente a taxa de deteção de ataques, mas tem como consequência direta o elevado número de falsos positivos em pedidos legítimos complexos (como transações JSON e parâmetros codificados). Adicionalmente, estudos de desempenho focados no impacto de latência demonstraram que a inspeção profunda de pacotes e a avaliação acumulativa de pontuações de anomalia (*Anomaly Scoring*) introduzem uma degradação mensurável no tempo médio de resposta do servidor Web sob condições de elevado tráfego concorrente.

A Tabela 2 sintetiza os principais eixos de análise presentes na literatura e a proposta para este projeto.

Tabela 2: Comparação entre trabalhos relacionados da literatura e a proposta do projeto.

| **Trabalho / Estudo** | **Solução WAF** | **Alvo de Teste** | **Vulnerabilidades** | **Métricas Avaliadas** |
|---|---|---|---|---|
| Estudos de Evasão e Robustez | ModSecurity / CRS | Aplicações sintéticas e testes laboratoriais | SQLi avançado e mutações sintáticas | Taxa de deteção (*True Positive Rate*) e evasão (*Bypass*). |
| Avaliações de Benchmark | WAFs comerciais vs. ModSecurity | Ambientes vulneráveis | OWASP Top 10 generalista | Cobertura de vulnerabilidades e alertas gerados. |
| Análise de Sobrecarga de Rede | ModSecurity em Apache/Nginx | Servidores Web de teste | Tráfego HTTP geral | Latência, tempo de resposta e pedidos por segundo. |
| **Proposta** | **ModSecurity v3 + CRS em Nginx** | **DVWA (Ambiente em Docker)** | **SQLi (In-Band, Blind) e XSS (Reflected, Stored)** | **Matriz de Confusão (Precisão, Recall, F1) e Latência ($\Delta L$).** |

# 4 Formulação Teórica e Métricas Matemáticas

Para validar formalmente o comportamento do WAF, recorre-se à matriz de confusão e às métricas padronizadas de eficácia em sistemas de deteção:

- **True Positives ($TP$):** Ataques reais que o WAF bloqueou corretamente.
- **False Positives ($FP$):** Pedidos legítimos bloqueados incorretamente.
- **True Negatives ($TN$):** Pedidos legítimos que foram permitidos pelo WAF.
- **False Negatives ($FN$):** Ataques que contornaram a deteção e foram aceites.

## 4.1 Métricas de Deteção

A **Precisão** traduz a fração de bloqueios efetuados que eram efetivamente maliciosos:

$$\text{Precisão} = \frac{TP}{TP + FP} \tag{1}$$

A **Sensibilidade** mede a percentagem do total de ataques que o sistema conseguiu travar:

$$\text{Sensibilidade} = \frac{TP}{TP + FN} \tag{2}$$

O **F1-Score** consiste na média harmónica entre a precisão e a sensibilidade:

$$\text{F1-Score} = 2 \times \frac{\text{Precisão} \times \text{Sensibilidade}}{\text{Precisão} + \text{Sensibilidade}} \tag{3}$$

A **Especificidade** (Taxa de Verdadeiros Negativos) avalia a capacidade do WAF permitir corretamente o tráfego legítimo:

$$\text{Especificidade} = \frac{TN}{TN + FP} \tag{4}$$

Uma vez que os conjuntos de dados nos testes de segurança apresentam frequentemente desbalanceamento substancial, recorre-se à **Acurácia Balanceada** (*Balanced Accuracy*), evitando distorções provocadas pela acurácia simples:

$$\text{BA} = \frac{\text{Sensibilidade} + \text{Especificidade}}{2} = \frac{1}{2}\left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP}\right) \tag{5}$$

## 4.2 Métricas de Desempenho e Latência

A sobrecarga de processamento introduzida pela inspeção profunda de pacotes é quantificada pela diferença do tempo médio de resposta ($\Delta L$) com o motor de inspeção ativo ($L_{\text{WAF}}$) contra o inativo ($L_{\text{Base}}$):

$$\Delta L = L_{\text{WAF}} - L_{\text{Base}} = \frac{1}{N}\sum_{i=1}^{N} t_i^{(\text{WAF})} - \frac{1}{M}\sum_{j=1}^{M} t_j^{(\text{Base})} \tag{6}$$

# 5 Metodologia e Repositório Online

## 5.1 Ambiente de Prova de Conceito

A arquitetura prática planeada para a Parte 2 integrará:

1. Um contentor **Nginx com ModSecurity v3** configurado como *Reverse Proxy*.
2. Um contentor com a aplicação vulnerável **DVWA** (*Damn Vulnerable Web Application*).
3. Ferramentas de injeção e teste de carga (*OWASP ZAP*, *sqlmap*, *cURL* e *Apache Benchmark*).

## 5.2 Repositório de Documentação (GitHub)

Para cumprir as diretrizes de armazenamento de resultados, todo o projeto encontra-se documentado no repositório da equipa: https://github.com/JoaOliveira21/Projeto_SRSI

# 6 Conclusões e Trabalho Futuro

A elaboração da parte teórica permitiu estruturar os conceitos de segurança em aplicações Web e estabelecer uma base de comparação sólida entre soluções de WAF. Foram colocadas as equações matemáticas que servirão de critério quantitativo de avaliação de desempenho e eficácia.

Na Parte 2 do trabalho prático, será implementado o ambiente virtualizado em Docker, procedendo-se aos testes de injeção sistemáticos, medição de latência com o Apache Benchmark e extração dos registos de auditoria do ModSecurity.

## Referências


[1] OWASP Foundation, "OWASP Top 10: The Ten Most Critical Web Application Security Risks," https://owasp.org/www-project-top-ten/, 2021.

[2] C. Folini and W. Blume, *OWASP ModSecurity Core Rule Set (CRS) Official Documentation*, OWASP Foundation, 2023.

[3] M. Alenezi and H. S. S. Al-Dhafer, "Effectiveness of Web Application Firewalls: A Comparative Study of Open-Source Solutions," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 11, no. 5, pp. 245–252, 2020.

[4] P. Sharma and R. K. Patel, "Detection and Mitigation of SQL Injection and Cross-Site Scripting Attacks Using ModSecurity and Machine Learning," in *2021 IEEE International Conference on Computing, Communication, and Intelligent Systems (ICCCIS)*, pp. 412–417, 2021.

[5] A. Varghese and T. Jacob, "Performance and Latency Evaluation of Web Application Firewalls in Cloud and Virtualized Environments," *Journal of Information Security and Applications*, vol. 48, pp. 102–111, 2019.